Day 4 — Lab 4.2: Metadata-Driven Multi-File Pipeline

1. Business scenario

Your organization receives multiple CSV files every day from different business systems.

Currently, the data engineering team creates a separate ADF pipeline for every file. This increases maintenance effort and makes onboarding new files difficult.

You are assigned to build a metadata-driven pipeline that:

- Reads file-processing instructions from a metadata CSV.
- Uses a Lookup activity to retrieve the file list.
- Uses a ForEach activity to process each file.
- Copies each CSV file from ADLS Gen2 into Parquet format.
- Uses the metadata to determine the output filename.

Business requirement: Adding a new file should require only a new metadata entry and the source CSV—not a new pipeline.

2. Expected architecture

ADLS Gen2 — Metadata

file_metadata.csv

ADF Lookup Activity

ForEach Activity

Iterates through metadata records

Copy Activity

CSV → Parquet for each file

ADLS Gen2 — Processed

customers.parquet, products.parquet, transactions.parquet

3. Resources required

Reuse the resources from Lab 4.1:

Resource	Name
Storage Account	`stadfuser01d4`
Data Factory	`adf-user01-d4`
Linked Service	`LS_ADLS_D4`

No new Azure services are required.

4. Input dataset design

We will use three source CSV files and one metadata CSV.

File	Records	Purpose

customers.csv	30	Customer information
products.csv	6	Product information
transactions.csv	60	Transaction information
file_metadata.csv	3 mappings	Controls the pipeline

The metadata file will contain:

```
None
sourceFile,targetFile
customers.csv,customers.parquet
products.csv,products.parquet
transactions.csv,transactions.parquet
```

The actual CSV files will be generated in our final artifact pack after all three labs are finalized.

5. Prepare the ADLS Gen2 folders

Open:

Azure Portal → Storage Account → Containers

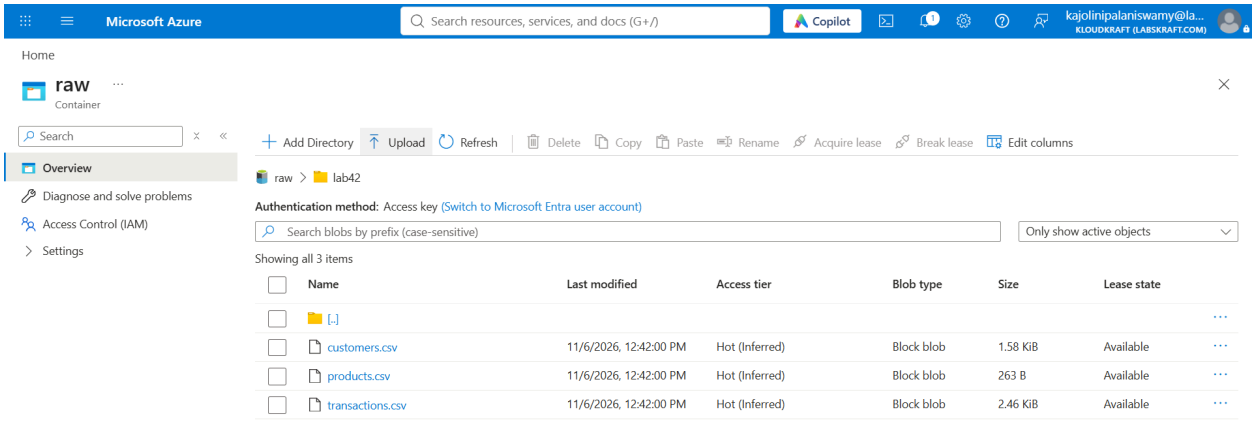
Create the following folders:

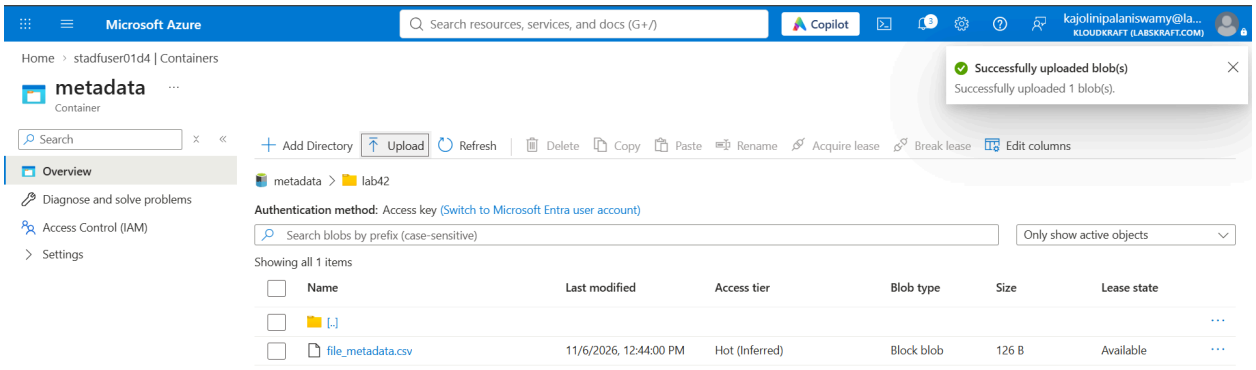
None

```
raw/
└── lab42/
    ├── customers.csv
    ├── products.csv
    └── transactions.csv

metadata/
└── lab42/
    └── file_metadata.csv

processed/
└── lab42/
```





Upload the files to their respective folders when the datasets are available.

Checkpoint 1: Confirm that the three source files and metadata file are present in the correct locations.

6. Create the metadata source dataset

Open Azure Data Factory Studio → Author.

1. Select + → Dataset.
2. Select Azure Data Lake Storage Gen2.
3. Select DelimitedText.
4. Select Continue.

Configure:

Setting	Value
Name	`DS_D4_FileMetadata`
Linked service	`LS_ADLS_D4`
Filesystem	`metadata`
Directory	`lab42`
File	`file_metadata.csv`
First row as header	Enabled
Column delimiter	Comma
Encoding	UTF-8

Select OK.

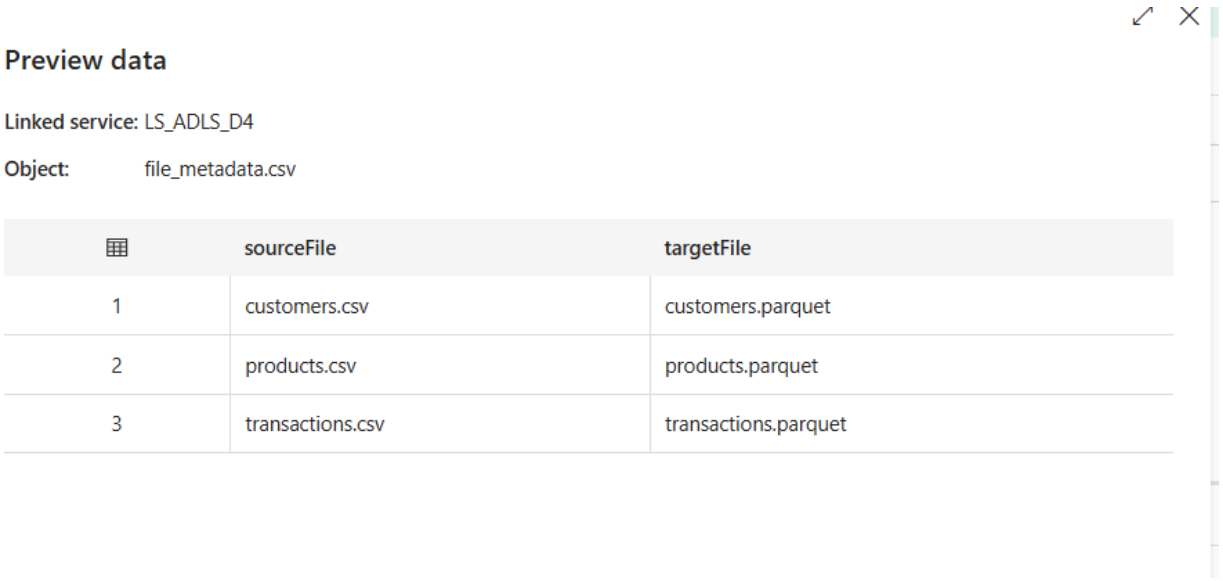
Open the dataset and select Preview data.

Expected columns:

- `sourceFile`

- `targetFile`

Expected records: 3.



Preview data

Linked service: LS_ADLS_D4

Object: file_metadata.csv

	sourceFile	targetFile
1	customers.csv	customers.parquet
2	products.csv	products.parquet
3	transactions.csv	transactions.parquet

7. Create a parameterized source dataset

Instead of creating three separate CSV datasets, we will create one reusable dataset.

Navigate to Author → + → Dataset.

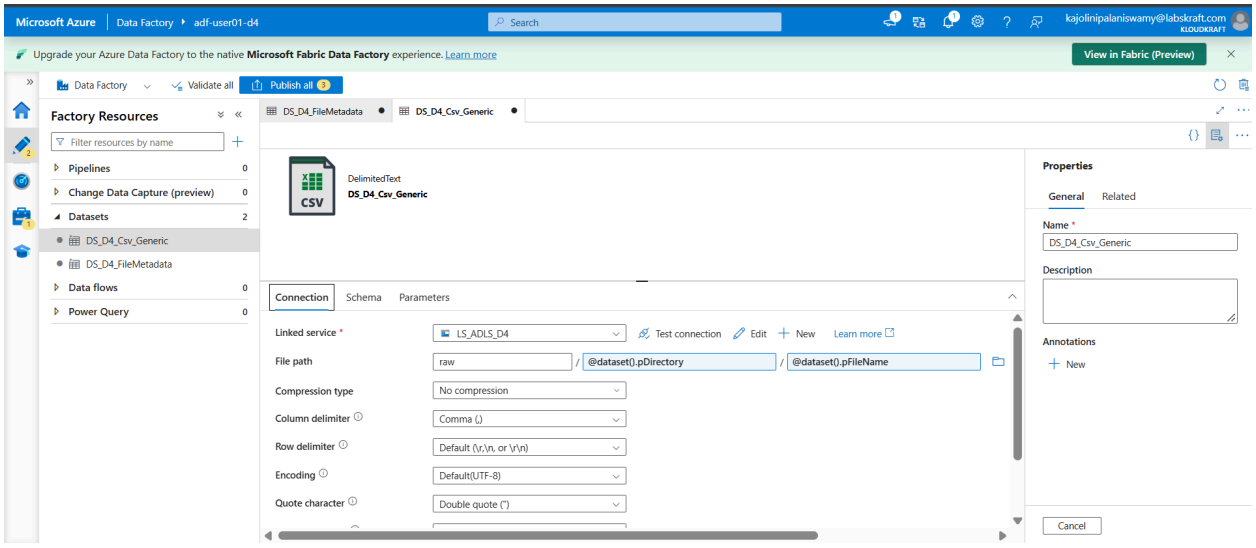
Select:

- Azure Data Lake Storage Gen2
- DelimitedText

Configure the dataset:

Setting	Value
Name	`DS_D4_Csv_Generic`

Linked service	`LS_ADLS_D4`
Filesystem	`raw`
Directory	Dynamic
File	Dynamic
First row as header	Enabled
Column delimiter	Comma
Encoding	UTF-8

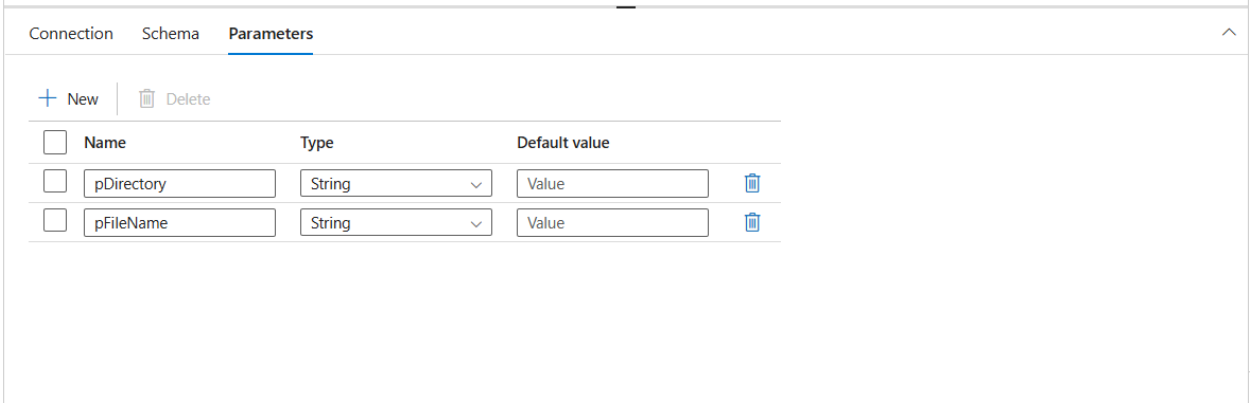


Add dataset parameters

Open the Parameters tab.

Add:

Parameter	Type
`pDirectory`	String
`pFileName`	String



Now open the Connection tab.

Set the directory field using Add dynamic content:

```
None
@dataset().pDirectory
```

Set the file name field using:

```
None
@dataset().pFileName
```

Select OK.



The same dataset can now read any CSV file from the configured raw filesystem, provided it follows the expected CSV format.

8. Create a parameterized destination dataset

Repeat the dataset creation process.

Select:

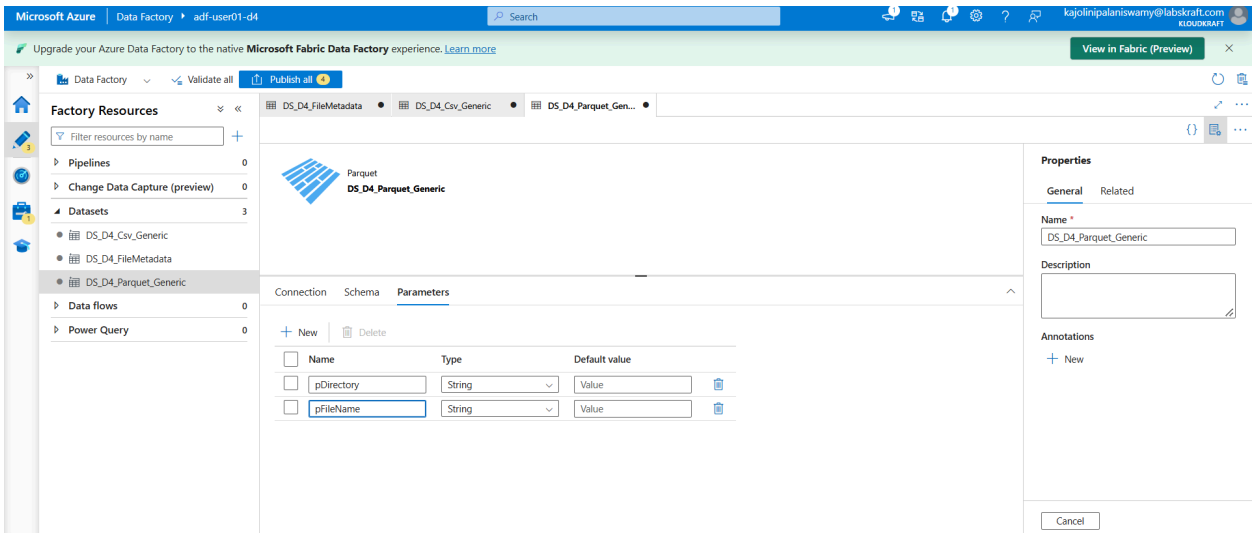
- Azure Data Lake Storage Gen2
- Parquet

Configure:

Setting	Value
Name	`DS_D4_Parquet_Generic`
Linked service	`LS_ADLS_D4`
Filesystem	`processed`
Directory	Dynamic
File	Dynamic

Add dataset parameters

Parameter	Type
`pDirectory`	String
`pFileName`	String



In the Connection tab, configure:

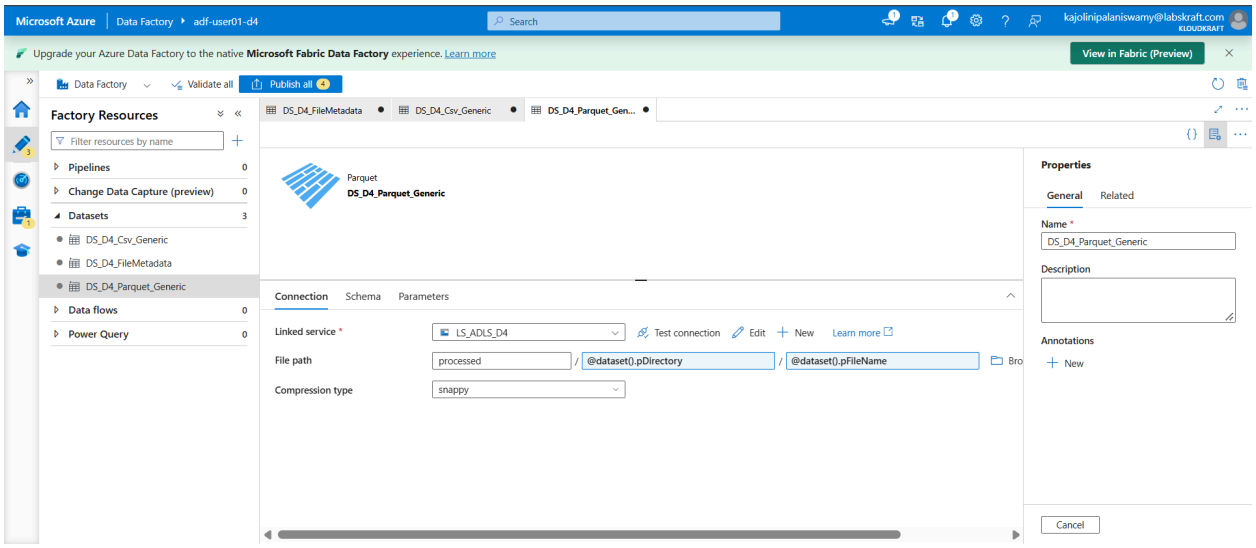
Directory:

```
None
@dataset().pDirectory
```

File:

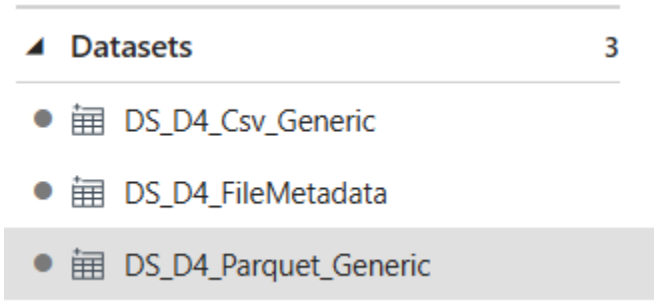
```
None
@dataset().pFileName
```

Select OK.



Checkpoint 2: We now have three datasets:

- DS_D4_FileMetadata
- DS_D4_Csv_Generic
- DS_D4_Parquet_Generic



One metadata dataset and two reusable datasets.

9. Create the metadata-driven pipeline

Navigate to Author → + → Pipeline.

Set the pipeline name:

```
None
PL_D4_02_MetadataDriven
```

Add Lookup activity

1. Open Activities.
2. Expand General.
3. Drag Lookup onto the pipeline canvas.
4. Set its name to:

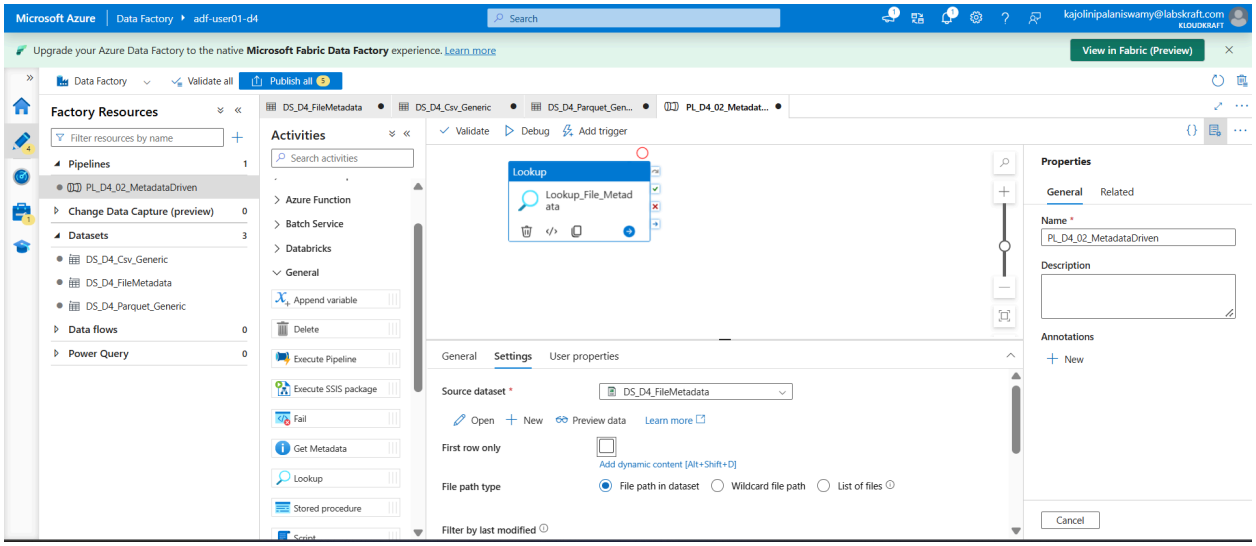
```
None
Lookup_File_Metadata
```

Open the Settings tab.

Configure:

Setting	Value
Source dataset	DS_D4_FileMetadata
First row only	Unchecked / False

Important: First row only must be disabled because we need all three metadata records, not just the first record.



10. Configure the ForEach activity

From Activities:

1. Expand Iteration & Conditionals.
2. Drag ForEach onto the pipeline canvas.
3. Connect Lookup to ForEach using the green Success dependency.
4. Rename the activity:

```
None
ForEach_File
```

Open the Settings tab.

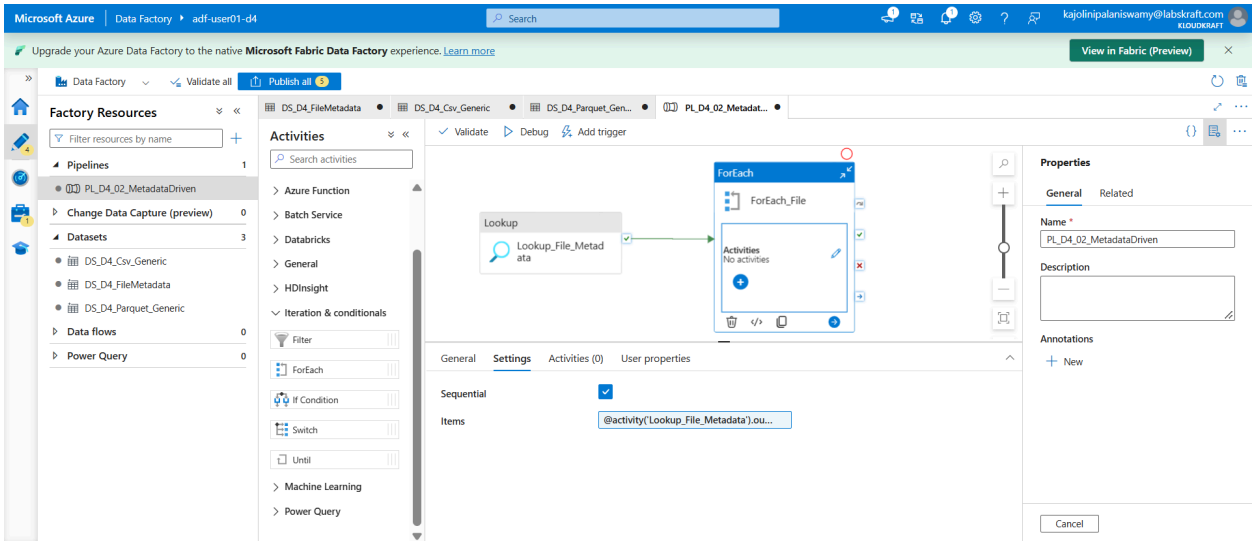
Under Items, select Add dynamic content.

Enter:

```
None
@activity('Lookup_File_Metadata').output.value
```

Set Sequential to Enabled for this first execution.

This ensures files are processed one at a time, making validation and troubleshooting easier.



11. Add Copy Activity inside ForEach

Select the ForEach activity.

1. Select the pencil icon to open its inner activity canvas.
2. Select Add activity.
3. Expand Move & Transform.
4. Drag Copy data into the ForEach canvas.
5. Rename it:

```
None
Copy_Current_File
```

Configure the Source

Open the Copy Activity → Source tab.

Select:

None

```
DS_D4_Csv_Generic
```

Under Dataset properties, configure:

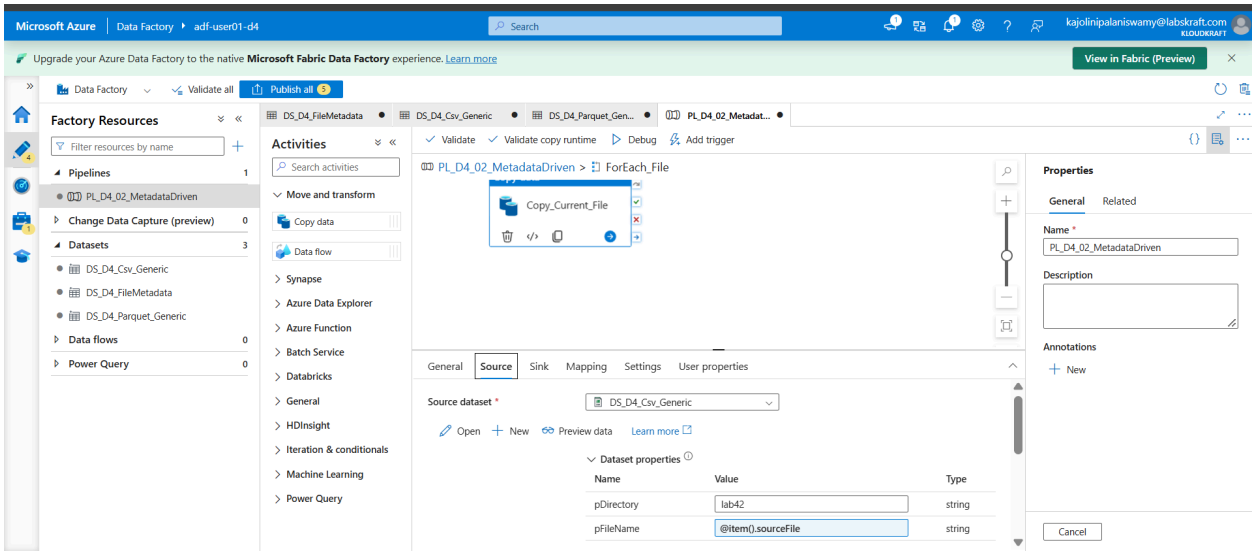
Parameter	Value
`pDirectory`	`lab42`
`pFileName`	Dynamic content

For `pFileName`, enter:

None

```
@item().sourceFile
```

This reads the source filename from the current metadata record.



Configure the Sink

Open the Sink tab.

Select:

```
None
DS_D4_Parquet_Generic
```

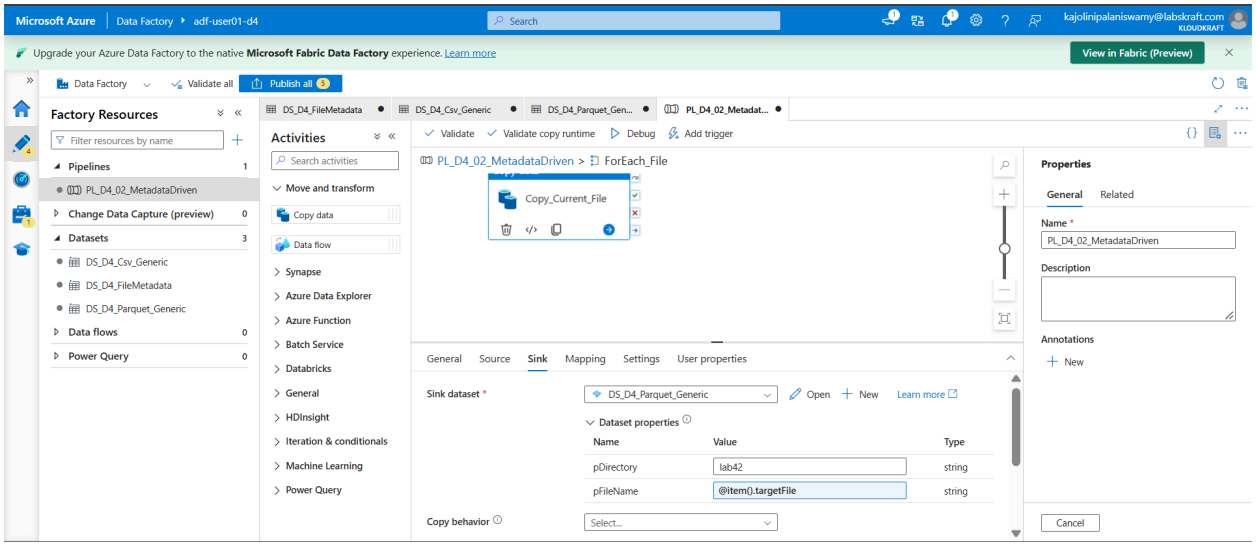
Configure:

Parameter	Value
`pDirectory`	`lab42`
`pFileName`	Dynamic content

For `pFileName`, enter:

```
None
@item().targetFile
```

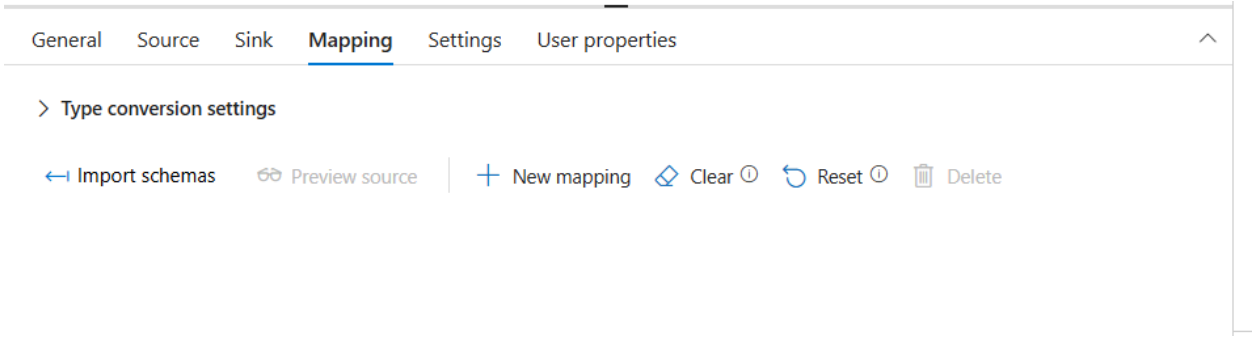
This determines the destination filename from the metadata record.



Configure mapping

Open the Mapping tab.

- Do not import and lock a fixed schema from only one source file.
- Leave automatic column mapping in place for this exercise.
- The three CSV files have different column structures, so each iteration must use the schema of its own source file.



Select the inner pipeline's back arrow to return to the main pipeline.

Your completed pipeline should have this structure:

Lookup_File_Metadata

On Success

ForEach_File

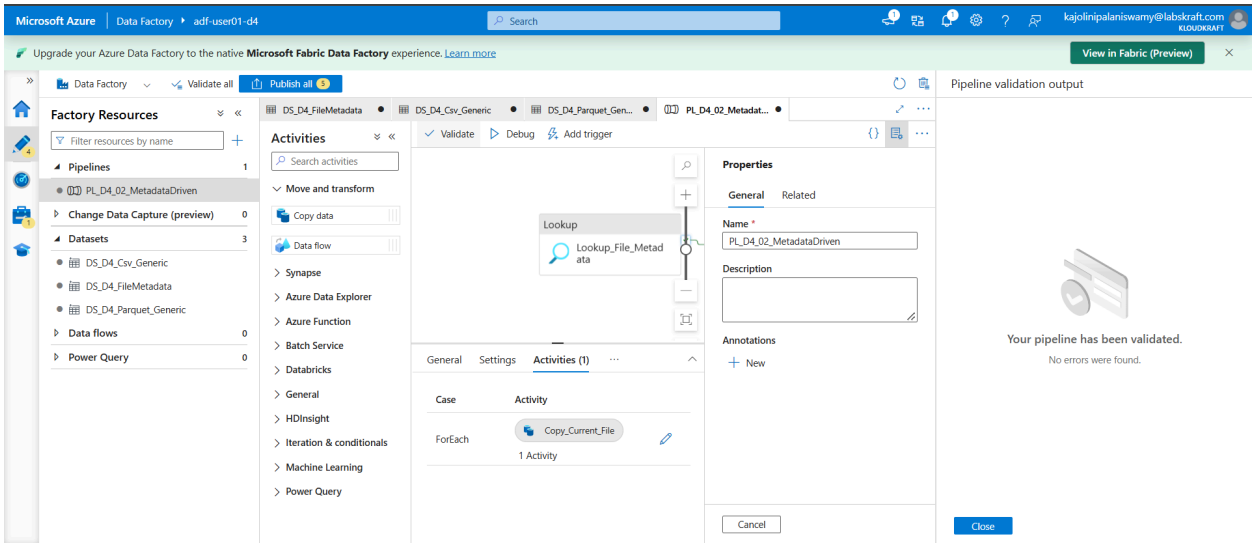
Items: Lookup output.value

Copy_Current_File

CSV → Parquet

12. Validate and execute the pipeline

1. Select Validate.
2. Resolve any validation errors.



3. Select Debug.
4. Wait for execution to complete.
5. Open the Output panel.
6. Select the ForEach activity and inspect its iterations.

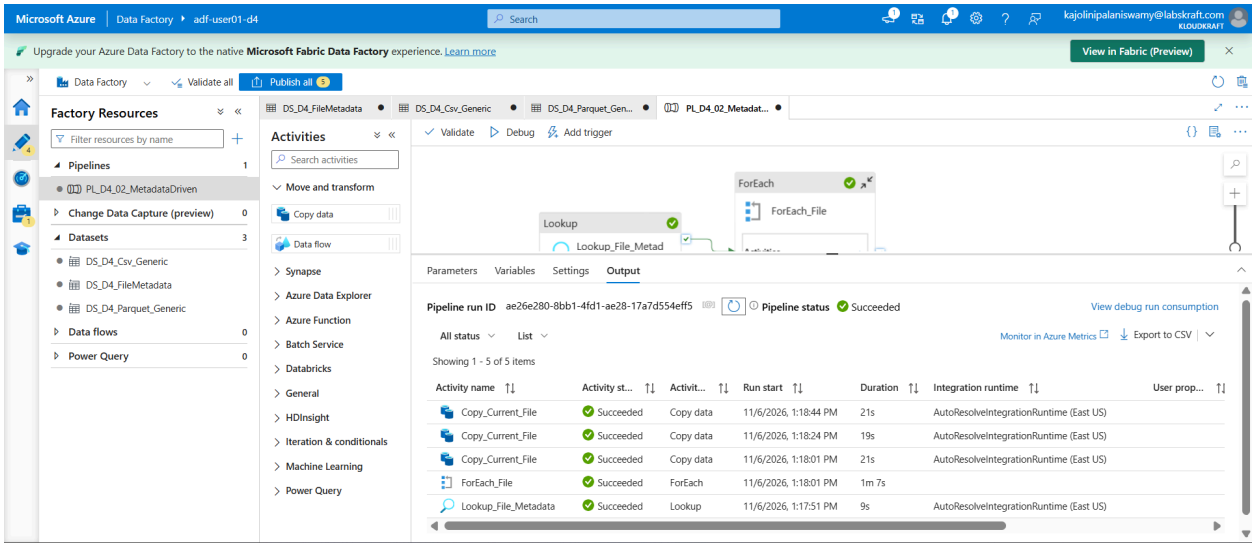
Expected results:

Iteration	Source	Destination	Expected rows

1	customers.csv	customers.parquet	30
2	products.csv	products.parquet	6
3	transactions.csv	transactions.parquet	60

Expected result

Lookup returns three records, ForEach executes three iterations, and all three Copy activities succeed.



Output

Copy to clipboard


```
{
    "count": 3,
    "value": [
        {
            "sourceFile": "customers.csv",
            "targetFile": "customers.parquet"
        },
        {
            "sourceFile": "products.csv",
            "targetFile": "products.parquet"
        },
        {
            "sourceFile": "transactions.csv",
            "targetFile": "transactions.parquet"
        }
    ],
    "effectiveIntegrationRuntime": "AutoResolveIntegrationRuntime (East US)",
    "billingReference": {
        "activityType": "PipelineActivity",
        "billableDuration": [
            {
                "meterType": "AzureIR",
                "duration": 0.016666666666666666,
                "unit": "Hours"
            }
        ],
        "totalBillableDuration": [
            {
                "meterType": "AzureIR",
                "duration": 0.016666666666666666,
                "unit": "Hours"
            }
        ]
    },
    "durationInQueue": {
        "integrationRuntimeQueue": 0
    }
}
```

13. Verify the output in ADLS Gen2

Navigate to:

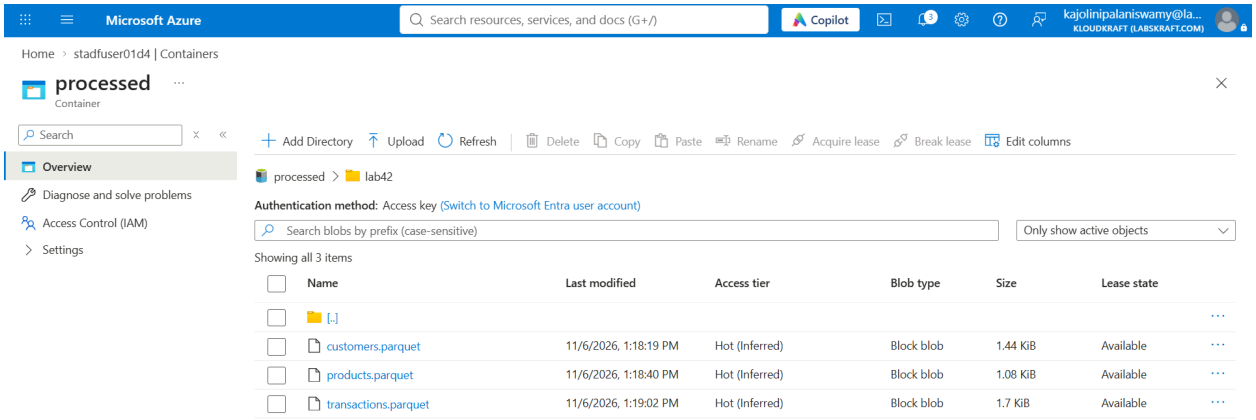
Azure Portal → Storage Account → Containers → processed → lab42

Expected output:

```
None
processed/
└── lab42/
    ├── customers.parquet
    ├── products.parquet
    └── transactions.parquet
```

Check the Copy Activity metrics for each iteration.

Output	Expected row count
customers.parquet	30
products.parquet	6
transactions.parquet	60
Total	96



14. Test the metadata-driven capability

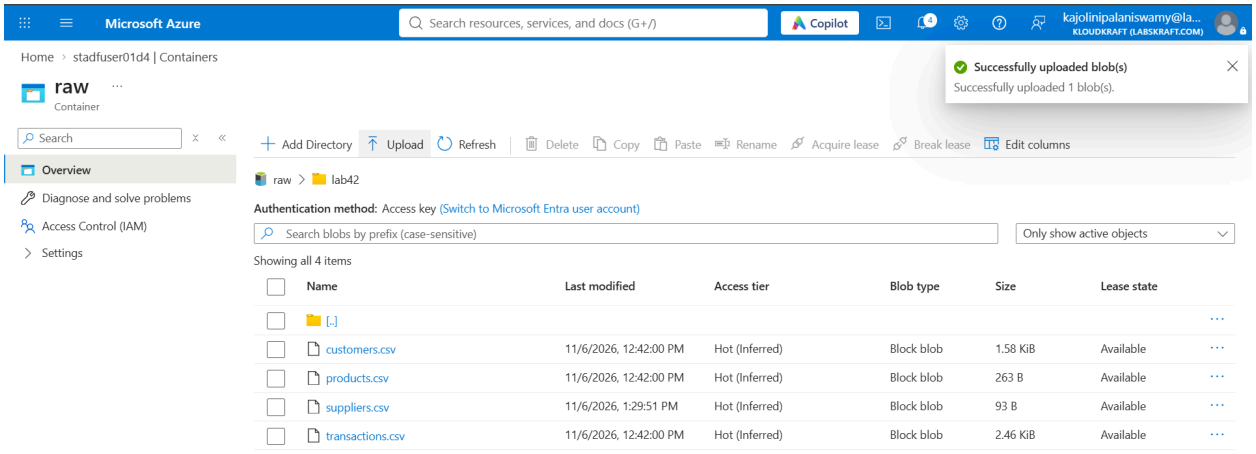
Now demonstrate why we built this pipeline.

Suppose the business team provides another file called:

```
None
suppliers.csv
```

To onboard it:

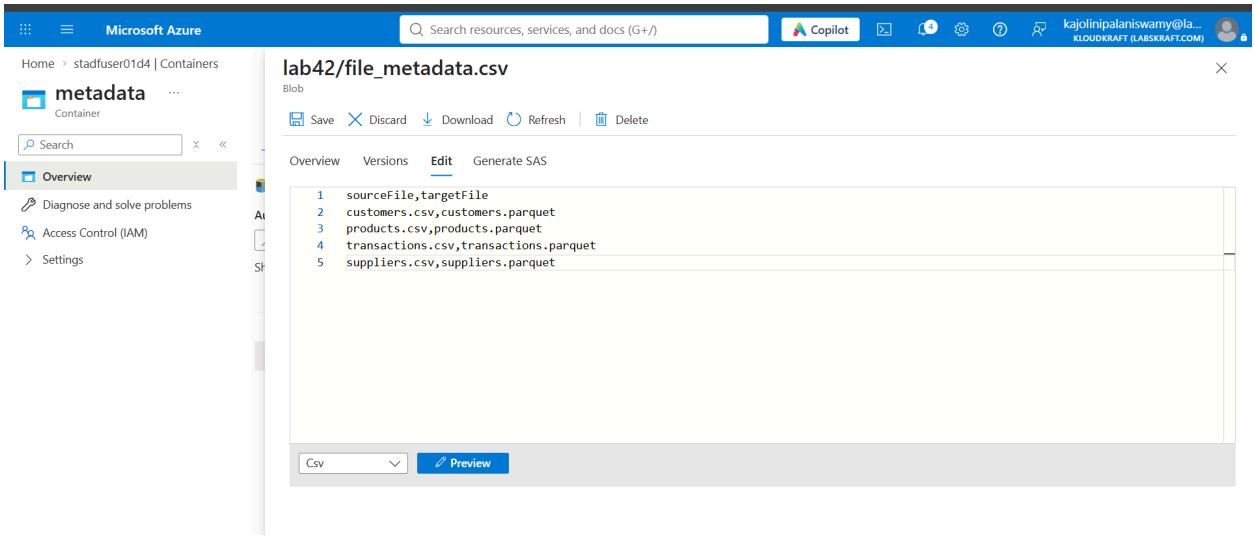
1. Upload `suppliers.csv` into `raw/lab42/`.



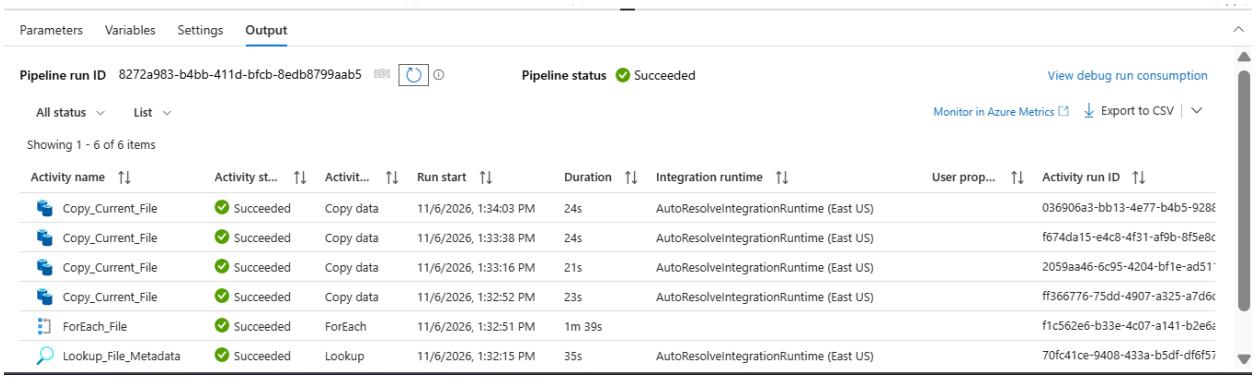
2. Add a new row to `file_metadata.csv`:

```
None
suppliers.csv,suppliers.parquet
```

3. Upload the updated metadata file, replacing the previous version.



4. Run the same pipeline again.

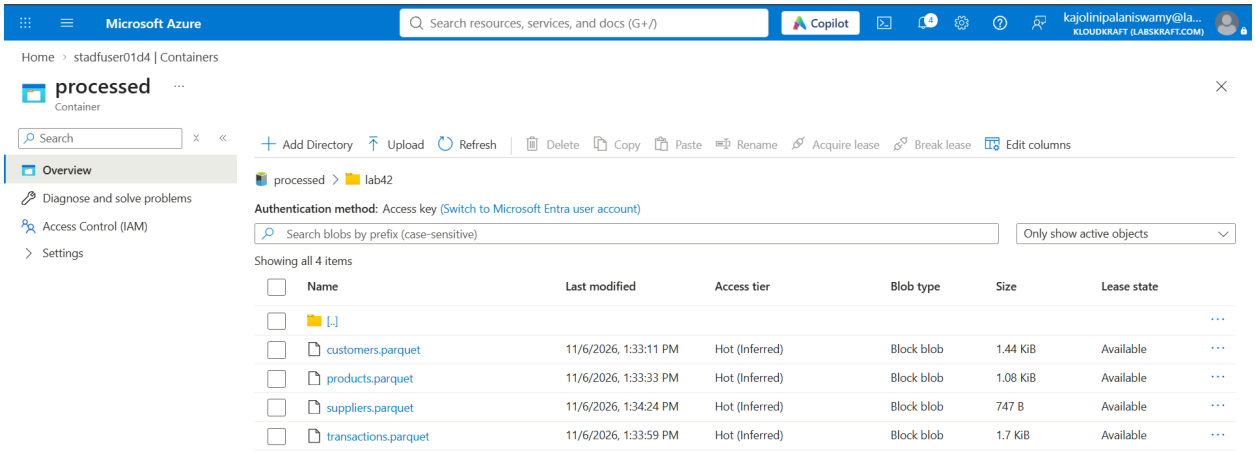


Expected output:

```
None
processed/lab42/
├── customers.parquet
├── products.parquet
```

```
├── transactions.parquet
└── suppliers.parquet
```

No new pipeline or Copy Activity is required, provided the new file follows the expected CSV format and the same folder conventions.



15. Evidence collection

Capture these screenshots:

- ADLS Gen2 raw folder containing the three CSV files.
- Metadata CSV preview showing three records.
- Parameterized source dataset configuration.
- Parameterized destination dataset configuration.
- Lookup activity configuration.
- Lookup output showing three records.
- ForEach activity configuration.
- Inner Copy Activity source and sink parameter settings.
- Successful pipeline execution and three iterations.
- Processed folder containing all three Parquet files.

16. Troubleshooting

Problem	What to check
Lookup returns only one row	First row only must be disabled.
ForEach executes zero iterations	Verify the Lookup output expression.
Source file not found	Check `raw/lab42` and exact filename.
Output file has wrong name	Check `@item().targetFile`.
Copy activity fails	Check source/sink dataset parameters and linked service.
Column mapping errors	Remove fixed schema mappings and validate automatic mapping for each file.
Storage access denied	Check Data Factory managed identity and Storage Blob Data Contributor role.

17. Lab 4.2 completion criteria

Validation	Expected result
Metadata dataset	Preview successful
Generic CSV dataset	Parameterized
Generic Parquet dataset	Parameterized
Lookup	3 records
ForEach	3 iterations
Copy activities	All successful
Total records copied	96
Output files	3 Parquet files
New metadata entry test	New file processed without another pipeline